

CYBR 437 Secure Coding
Winter 2026
Lab - 8
Release date: February 24, 2026
Due date: March 2, 2026, at 11:59 pm

Introduction

In Part 1 of this lab, you will investigate specific integer operation issues in C with hands-on exercises. You will identify potential problems in the given C code and revise the code to address these problems. You will need to use the header file `limits.h` in C that defines macros to specify the minimum and maximum values of fundamental data types for the specific system and compiler implementation. It allows programmers to use these limits in a portable way without hard-coding system-specific values (and you will not be hard-coding). You can include the said header file in a C program using the preprocessor directive `#include <limits.h>`.

In Part 2 of the lab, you will examine the provided C code to identify any potential memory management issues and provide rectifications.

Part 1 (40 points)

Task 1

The following function has an integer overflow vulnerability during addition.

```
void fun1 (int a, int b) {  
    int sum;  
    sum = a + b;  
    printf("Result = %d\n", sum);  
}
```

- Demonstrate the vulnerability by writing a program and calling this function with two argument values that cause an overflow. The values of the arguments themselves should be within their permissible range. Explain in detail why you chose these values.
- Then rewrite the function to either print a correct result or print an error message if there is a possibility of an overflow.
- Finally, show the execution of the program with the previous values of the arguments that caused an overflow, and with two other values that do not cause an overflow. Explain the output.

Task 2

The following function has an integer overflow vulnerability during subtraction.

```
void fun2 (int x, int y) {
    int diff;
    diff = x - y;
    printf("Result = %d\n", diff);
}
```

- Demonstrate the vulnerability by writing a program and calling this function with two argument values that cause an overflow. The values of the arguments themselves should be within their permissible range. Explain in detail why you chose these values.
- Then rewrite the function to either print a correct result or print an error message if there is a possibility of an overflow.
- Finally, show the execution of the program with the previous values of the arguments that caused an overflow, and with two other values that do not cause an overflow. Explain the output.

Part 2 (20 points)

You need not write or run programs for this part. Simply provide the explanations as asked.

Task 3

Identify any memory management errors in the following code and explain how you will rectify them.

```
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char *return_val = 0;
    const size_t bufsize = strlen(argv[0]) + 1;
    char *buf = (char *)malloc(bufsize);
    if (!buf) {
        return EXIT_FAILURE;
    }
    free(buf);
    strcpy(buf, argv[0]);
    return EXIT_SUCCESS;
}
```

Task 4

Identify any memory management errors in the following code and explain how you will rectify them.

```
#include <stdlib.h>

int fun(void) {
    char *text_buffer = (char *)malloc(32);
    if (text_buffer == NULL) {
        return -1;
    }
    return 0;
}
```

Task 5

Identify any memory management errors in the following code and explain how you will rectify them.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    ptr = (int*)malloc(sizeof(int));
    *ptr = 10;
    printf("%d\n", *ptr);
    free(ptr);
    printf("%d\n", *ptr);
    return 0;
}
```

Submission Instructions

Your write-up submission should include:

- The code, explanations, and screenshots of the outputs for the tasks in Part 1
- Explanations for the tasks in Part 2